Pitch: Limites e Teoria das Filas

Roteiro para apresentação de 10 minutos

ABERTURA

0:00 – 1:30

A fila que você não vê

Imagine que você entra numa padaria. Há três atendentes, mas de repente chegam cinquenta pessoas ao mesmo tempo. O que acontece? Algumas esperaram, outras foram embora, e a padaria provavelmente perdeu vendas. Agora substitua a padaria por um servidor web — e multiplique esse problema por milhões de requisições por segundo.

Hoje vou mostrar como um conceito matemático de 300 anos de idade — o limite — ajuda engenheiros a resolver esse problema de forma elegante.

> *Dica de apresentação: comece com o exemplo físico da padaria no slide — cria ancoragem visual antes do abstrato.*

CONCEITO

1:30 – 3:00

O que é um limite?

Em matemática, um limite descreve o comportamento de uma função à medida que uma variável se aproxima de um valor — sem necessariamente atingi-lo. Formalmente, dizemos que o limite de f(x) quando x tende a c é L quando os valores de f(x) se aproximam arbitrariamente de L.

> *O limite não pergunta 'o que acontece exatamente em c?'. Ele pergunta: 'para onde o sistema caminha quando se aproxima de c?'*

Essa distinção é poderosa: sistemas reais raramente operam em pontos exatos — eles operam em aproximações. Taxa de chegada de pacotes, tempo de resposta, utilização de CPU: tudo são funções que tendem a comportamentos-limite, nunca estáticos.

> *Dica de apresentação: Use um gráfico simples de f(x) → L com x → c. Mantenha a notação visual, sem equações densas.*

TEORIA DAS FILAS

3:00 – 6:00

Da matemática pura à matemática das filas

A Teoria das Filas — formalizada por A.K. Erlang no início do século XX para dimensionar centrais telefônicas — modela sistemas onde há chegadas, espera e atendimento. O modelo mais clássico é o M/M/1: chegadas seguem processo de Poisson, atendimentos seguem distribuição exponencial, e há um único servidor.

Aqui entra o limite de forma concreta. Definimos ρ = λ/μ como a intensidade de tráfego: a razão entre a taxa de chegada (λ) e a taxa de serviço (μ). O sistema só é estável quando ρ < 1. Quando ρ → 1, o tempo médio de espera na fila tende ao infinito:

> *lim (ρ → 1⁻) W = ∞ — o sistema colapsa antes de ρ chegar a 1.*

Esse é o limite crítico de operação: cruzá-lo não significa falha imediata, mas degradação exponencial. Um servidor com 95% de utilização não está 'quase cheio' — está geometricamente próximo do colapso.

> *Dica de apresentação: Mostre a curva de W(ρ): ela é o melhor argumento visual — cresce lentamente até ~0.7 e explode depois.*

O PROBLEMA

6:00 – 9:00

Controle de tráfego de servidor: onde a teoria encontra a produção

Em infraestrutura moderna, todo servidor HTTP, banco de dados ou API enfrenta o mesmo problema: como aceitar mais requisições do que consegue processar, sem derrubar o serviço?

A resposta ingênua é 'aumentar a capacidade'. A resposta baseada em teoria das filas é: definir limites de operação antes que o sistema cruze o ponto crítico. Três estratégias diretas emergem dessa análise:

Rate limiting — impor um teto em λ, garantindo que a intensidade ρ permaneça em zona segura. Ferramentas como token bucket e leaky bucket são implementações de controle de limite de taxa de chegada.

Circuit breaker — quando o sistema detecta que está se aproximando de ρ crítico (via latência crescente ou fila de espera), ele rejeita requisições proativamente. É o limite agindo como mecanismo de proteção: preferimos negar 5% das requisições a deixar 100% degradarem.

Backpressure — em vez de descartar, o servidor sinaliza para os clientes que devem desacelerar. A taxa de chegada é limitada pelo próprio limite de capacidade a jusante — o sistema se auto-regula em torno de ρ < 1.

> *Dica de apresentação: Use um diagrama de fluxo: Cliente → Fila → Servidor. Mostre onde cada estratégia age. Mantenha simples.*

CONCLUSÃO

9:00 – 10:00

Limites como design, não como emergência

O ponto central é este: sistemas resilientes não reagem ao colapso — eles são projetados em torno dos seus limites. A teoria das filas nos dá a linguagem matemática para definir esses limites com precisão, e o conceito de limite nos diz o que acontece quando nos aproximamos deles.

Engenharia de sistemas, no fundo, é a arte de operar confortavelmente longe do infinito.

O Código por trás da Curva

Para tornar o conceito tangível, simulamos um sistema M/M/1 real usando SimPy, uma biblioteca Python de simulação por eventos discretos. O código gera requisições seguindo uma distribuição de Poisson — exatamente como o modelo teórico assume — e processa cada uma com um único servidor de capacidade fixa (μ = 10 req/s).

```
TAXA_ATENDIMENTO_MU = 10.0  # μ: requisições processadas por segundo


def gerador_requisicoes(env, servidor, taxa_chegada, mu, metricas):
    while True:
        yield env.timeout(random.expovariate(taxa_chegada))
        env.process(processar_requisicao(env, servidor, mu, metricas, req_id))
```

A simulação roda nove cenários distintos, variando ρ de 0,10 até 0,99. Para cada cenário, medimos o tempo médio de espera W e o tamanho médio da fila — e comparamos com o valor exato dado pela fórmula analítica W = 1/(μ − λ).

```
cargas_rho = np.array([0.10, 0.30, 0.50, 0.70, 0.80, 0.90, 0.95, 0.97, 0.99])
lambdas    = cargas_rho * TAXA_ATENDIMENTO_MU


def calcular_w_teorico(lam, mu):
    if lam >= mu:
        return float('inf')
    return 1.0 / (mu - lam)  # W = 1/(μ − λ)
```

O resultado confirma o que o limite prevê: até ρ ≈ 0,70, o sistema se comporta de forma previsível e controlada. A partir daí, pequenos aumentos em λ produzem aumentos desproporcionais em W. Com ρ = 0,95, o tempo de espera é quase seis vezes maior do que com ρ = 0,50 — mesmo com apenas 45% mais carga.

```
servidor = simpy.Resource(env, capacity=1)  # M/M/1: um único servidor


def processar_requisicao(env, servidor, mu, metricas, req_id):
    chegada = env.now
    with servidor.request() as req:
        yield req
        espera = env.now - chegada
        metricas.tempos_espera.append(espera)

        tempo_servico = random.expovariate(mu)
        yield env.timeout(tempo_servico)
```

Isso não é uma falha do sistema. É exatamente o que a matemática anuncia: a função diverge antes de λ chegar a μ. O código apenas torna visível o que o limite já descrevia.